

2024 国际大学生程序设计竞赛亚洲区域赛（昆明站）

正式赛



题目列表

A	防毒
B	括号
C	金币
D	套娃
E	提取权重
F	花
G	最大公因数
H	扫描地平线
I	物品
J	又一个排序问题
K	密钥恢复
L	最后的机会：绝望线缕
M	矩阵构造

本试题册共 13 题，21 页。
如果您的试题册不完整，请立即通知志愿者。

2024 年 12 月 1 日

承办方



命题方



Problem A. 防毒

网络流入门课上，老师刚讲了一点，学生还没明白，老师养的小鸡就学会怎么做题了。老师不禁感叹：我们家禽流感真好呀。

—烧鸡块君

Hyrule 进入了流感季。

Hyrule 包含 n 座城市，这些城市由 m 条有向道路连接。所有城市都可以通过这些道路到达首都（编号为 1 的城市）。

流感季将持续 q 天。第 i 天中午，流感病毒将从编号为 a_i 的城市开始传播。病毒将会沿着道路向其他城市传播。病毒传播的速度很快，因此可以认为所有从 a_i 出发可以到达的城市都会立刻感染病毒。作为 Hyrule 的首都，编号为 1 的城市对国家至关重要。如果第 i 天的病毒传播到了首都，则会产生 b_i 的经济损失代价。

为了阻止病毒传播，Auru 每天晚上都可以在某座城市部署病毒过滤器（包括第一天前的晚上）。在编号为 i 的城市部署病毒过滤器会产生 c_i 的部署代价。病毒不能经由部署了病毒过滤器的城市传播，这座城市也不会感染病毒。如果病毒将从部署了病毒过滤器的城市开始传播，则病毒会直接消失而不感染任何城市。部署的病毒过滤器将会持续发挥作用，直到 Auru 在另一座城市部署新的过滤器。（也就是说，同时只能存在至多一个病毒过滤器。）

Auru 想知道：对于每个 $i = 1, 2, \dots, q$ ，仅考虑前 i 天的病毒，“经济损失代价 + 部署代价”的最小值是多少。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 10^4$)。每组测试数据的格式如下。

第一行包含三个整数 n, m 和 q ($2 \leq n \leq 10^5, n-1 \leq m \leq 2 \times 10^5, 1 \leq q \leq 10^5$)，分别代表城市的数量，道路的数量和流感季的持续天数。

接下来 m 行，每行两个整数 u_i 和 v_i ($1 \leq u_i, v_i \leq n$)，表示一条由城市 u_i 到城市 v_i 的有向道路。数据保证没有自环，但可能存在重边。保证从所有城市出发都可以到达首都。

接下来一行包含 n 个整数 $c_1, c_2, \dots, c_n$ ($1 \leq c_i \leq 10^9$)，表示在每座城市部署病毒过滤器的部署代价。

接下来 q 行，第 i 行包含两个整数 a_i 和 b_i ($2 \leq a_i \leq n, 1 \leq b_i \leq 10^9$)，表示第 i 天病毒开始传播的城市和该病毒传播到首都将产生的经济损失代价。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^5 ，所有测试数据的 m 之和不超过 2×10^5 ，所有测试数据的 q 之和不超过 10^5 。

Output

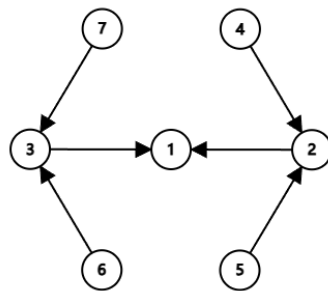
对于每组数据，输出一行 n 个整数，第 i 个整数表示仅考虑前 i 天的病毒时，“经济损失代价 + 部署代价”代价的最小值。

Example

standard input	standard output
3	2 3 4 4
7 6 4	5 100 102 202
2 1	10
3 1	
4 2	
5 2	
6 3	
7 3	
4 3 5 2 2 1 1	
4 2	
5 2	
6 2	
7 2	
5 6 4	
1 3	
3 2	
2 1	
4 2	
5 4	
2 5	
10000 10000 2 100 5	
5 1000	
4 1000	
3 1000	
4 1000	
4 4 1	
2 1	
3 1	
4 2	
4 3	
100 1 1 100	
4 10	

Note

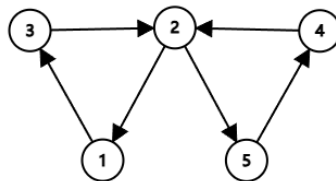
- 在第一组测试数据中：
- 前 1 天的最小代价是 2：第 0 天晚上，在 4 号城市部署病毒过滤器，部署代价为 2。
 - 前 2 天的最小代价是 3：第 0 天晚上，在 2 号城市部署病毒过滤器，部署代价为 3。
 - 前 3 天的最小代价是 4：第 0 天晚上，在 2 号城市部署病毒过滤器，部署代价为 3；第 2 天晚上，在 6 号城市部署病毒过滤器，部署代价为 1。
 - 前 4 天的最小代价是 4：第 0 天晚上，在 1 号城市部署病毒过滤器，部署代价为 4。



样例 1 示意图

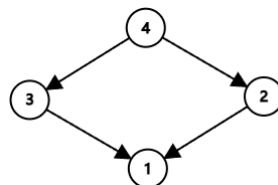
在第二组测试数据中：

- 前 1 天的最小代价是 5：第 0 天晚上，在 5 号城市部署病毒过滤器，部署代价为 5。
- 前 2 天的最小代价是 100：第 0 天晚上，在 4 号城市部署病毒过滤器，部署代价为 100。
- 前 3 天的最小代价是 102：第 0 天晚上，在 4 号城市部署病毒过滤器，部署代价为 100；第 2 天晚上，在 3 号城市部署病毒过滤器，部署代价为 2。
- 前 4 天的最小代价是 202：第 0 天晚上，在 4 号城市部署病毒过滤器，部署代价为 100；第 2 天晚上，在 3 号城市部署病毒过滤器，部署代价为 2；第 3 天晚上，在 4 号城市再次部署病毒过滤器，部署代价为 100。



样例 2 示意图

在第三组测试数据中，由于同时只能有一个病毒过滤器，不能通过在 2, 3 号城市同时部署病毒过滤器来阻止病毒传播。1, 4 号城市的部署代价大于病毒造成的经济损失代价，因此不部署病毒过滤器，最小代价为 10（经济损失代价）。



样例 3 示意图

Problem B. 括号

其实世界上本来只有三种括号。
后来大于号和小于号因为生活不易，每晚兼职扮演尖括号。

—炒鸡块君

Betty 有一个长度为 n 的括号序列 s ，其由八种括号组成：“() [] {} <>”。同时，她还有 m 个子括号序列，第 i 个子括号序列 t_i 是 s 的第 l_i 个字符到第 r_i 个字符连接成的序列，即 $t_i = s_{l_i} s_{l_i+1} \cdots s_{r_i}$ 。

Betty 想知道，从这 m 个子括号序列中取出若干对并将属于同一对的两个序列连接，最多能得到多少个合法的括号序列[†]？形式化地说，你需要找到尽可能多的二元组 (a_i, b_i) ，满足：

- 对于任何一个 1 到 m 中的整数 x ，其在 a 中出现的次数与在 b 中出现的次数之和小于等于 1。
- 对于任意一个二元组 (a_i, b_i) ， $t_{a_i} t_{b_i}$ 是一个合法的括号序列[†]。

[†] 一个括号序列 x 是合法的，当且仅当其满足以下条件之一：

- $|x| = 0$ ，即 x 是空串。
- $x = LyR$ ，其中 y 是一个合法的括号序列， L 和 R 分别是同一种括号的左括号和右括号（即 $L = “(”$ 且 $R = “)”$ ；或 $L = “[”$ 且 $R = “]”$ ；或 $L = “{”$ 且 $R = “}”$ ；或 $L = “<”$ 且 $R = “>”$ ）。
- $x = y_1 y_2$ ，其中 y_1 和 y_2 都是合法的括号序列。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 10^4$)。每组测试数据的格式如下。

第一行两个整数 n 和 m ($1 \leq n, m \leq 5 \times 10^5$)，分别表示 s 的长度和子括号序列的数量。

第二行包含一个字符串 s ($|s| = n$)，其仅包含 “() [] {} <>” 八种括号。

接下来 m 行，每行包含两个整数 l_i 和 r_i ($1 \leq l_i \leq r_i \leq n$)。

在每个测试文件内，保证所有测试数据的 n 之和不超过 5×10^5 ，所有测试数据的 m 之和不超过 5×10^5 。

Output

对于每组数据，输出一行一个整数，表示通过连接每对子括号序列能得到的合法括号序列数量的最大值。

Example

standard input	standard output
4	0
8 1	3
()[]{}<>	0
3 6	2
2 6	
) (	
1 1	
1 1	
1 1	
2 2	
2 2	
2 2	
6 2	
([]())	
1 3	
4 6	
22 8	
([{<<<<[]>>>>([]){()})	
3 8	
11 14	
1 10	
3 4	
19 22	
20 21	
17 20	
21 22	

Note

在第一组测试数据中, $t_1 = "[]\{"$ ”。尽管 t_1 已经是一个合法的括号序列, 没有其他子括号序列能与其配对, 所以输出是 0 。

在第二组测试数据中, 六个子括号序列包含三个 “(” 和三个 “)”, 可以拼出三个合法的括号序列 “()”。因此, 输出应该是 3。

在第三组测试数据中, 无法拼出任何合法的括号序列。

在第四组测试数据中, 一种可能的匹配方案是 t_1t_2 : “{<<<<” + “>>>>” 和 t_4t_5 : “{” + “{()}"”。

Problem C. 金币

政府决定裁撤海军规模，把省下来的金币直接发给海盗，这样能消灭的海盗比原来海军消灭的还多。

—煮鸡块君

海盗们刚刚夺得了一枚巨大的金币！

为了决定这枚金币的归属，他们决定采用以下办法选出金币的拥有者：

设目前剩余的海盗数量为 n 。所有海盗排成一队，然后位于第 $1, 1+k, 1+2k, \dots, 1+(\lceil \frac{n}{k} \rceil - 1)k$ 个的海盗被淘汰。重复这一操作，直到仅剩下一位海盗。最终剩下的海盗获得这枚金币。

Charlie 是海盗中最聪明的。他想知道，要想最终获得金币，一开始他应该站在第几位？

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 100$)。每组测试数据的格式如下。

第一行包含两个整数 n 和 k ($2 \leq n, k \leq 10^{12}$)，表示初始海盗的数量以及淘汰时用到的参数。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^{12} ，所有测试数据的 k 之和不超过 10^{12} 。

Output

对于每组数据，输出一行一个整数，表示最终获得金币的海盗在初始队列中的位置。

Example

standard input	standard output
4	4
6 2	8
8 3	8192
10000 2	1919805
1919810 114514	

Note

对于样例的第一组测试数据，每轮剩下的海盗在原始序列中的位置为：

- 初始状态：1, 2, 3, 4, 5, 6。
- 第一轮后：2, 4, 6。
- 第二轮后：4。

对于样例的第二组测试数据，每轮剩下的海盗在原始序列中的位置为：

- 初始状态：1, 2, 3, 4, 5, 6, 7, 8。
- 第一轮后：2, 3, 5, 6, 8。
- 第二轮后：3, 5, 8。
- 第三轮后：5, 8。
- 第四轮后：8。

Problem D. 套娃

小鸡玩有奖套圈游戏，套着套着
套到了一套套套娃。

—蒸鸡块君

David 刚刚获得了 n 个大小互不相同的俄罗斯套娃。他将这些套娃从左到右摆成了一排，第 i 个位置上包含一个大小为 a_i 的套娃。

设第 i 格上的最小套娃的大小为 l_i ，最大套娃的大小为 r_i 。两个相邻格子 i 和 $i+1$ 上的套娃能合并，当且仅当 $r_i < l_{i+1}$ 或 $r_{i+1} < l_i$ 。新的套娃将包含原有第 i 格和第 $i+1$ 格上的所有套娃，并被放置在第 i 格上。所有编号大于 $i+1$ 的格子上的套娃都将向左移动一格以填补空缺。

例如，当 $n = 4, a = [2, 1, 4, 3]$ 时，David 可以：

- 合并第 1 格和第 2 格上的套娃。此时剩余三个格子上的套娃大小分别为 $[(1, 2), (4), (3)]$ 。
- 合并第 2 格和第 3 格上的套娃。此时剩余两个格子上的套娃大小分别为 $[(1, 2), (3, 4)]$ 。
- 合并第 1 格和第 2 格上的套娃。此时所有套娃都被合并到了一个格子上。

在最优策略下，David 最多能执行多少次合并操作？

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 10^4$)。每组测试数据的格式如下。

第一行包含一个整数 n ($1 \leq n \leq 10^5$)，表示套娃的数量。

第二行包含 n 个整数 $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n, \forall i \neq j, a_i \neq a_j$)，表示初始状态下每个位置上的套娃的大小。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^5 。

Output

对于每组数据，输出一行一个整数，表示最多能执行的合并操作数。

Example

standard input	standard output
8	3
4	3
2 1 4 3	2
4	3
1 4 2 3	3
4	3
3 1 4 2	4
5	4
1 3 5 2 4	
5	
1 4 2 5 3	
5	
2 5 3 1 4	
6	
1 3 6 5 2 4	
6	
2 5 1 3 6 4	

Problem E. 提取权重

“由于 Emily 很忙”... 我咋感觉
Emily 挺闲的。

—烤鸡块君

这是一道交互题。

Emily 有一棵包含 n 个节点的树，树上编号为 i 的节点的权值是 w_i 。1 号节点是根节点，且根节点的权值为 0。

Emily 希望你猜出每个节点的权值。具体来说，你可以提出至多 n 个询问，第 i 个询问包含两个整数 u_i 和 v_i 。如果 u_i 到 v_i 的简单路径包含恰好 k 条边，Emily 会告诉你节点 u_i 到节点 v_i 的简单路径上所有节点的权值按位异或[†]的结果（包括端点）。否则，Emily 会回答你“-1”，表示她不想回答这个问题。Emily 很忙，因此只有在你提出了所有的问题后，她才会开始回答。

当然，对于某些情况，你可能无法通过不超过 n 个询问猜出所有点的权值。此时，你不应该发出任何询问，而是直接告诉 Emily 这是不可能做到的。

[†] 按位异或是指将两个二进制数字的每一位进行模 2 意义下的加法运算。例如： $(0011)_2 \oplus (0101)_2 = (0110)_2$ 。

Input

每个测试文件仅有一组测试数据。

第一行包含两个整数 n 和 k ($2 \leq n \leq 250, 1 \leq k \leq n-1$)，表示树的节点个数以及询问中提及的参数。

接下来 $n-1$ 行，每行包含两个整数 x_i 和 y_i ($1 \leq x_i, y_i \leq n$)，表示节点 x_i 和 y_i 之间连有一条边。

保证所有的边构成一棵树，且正确答案满足 $0 \leq w_i < 2^{30}$ 。

Interaction Protocol

首先，你需要判断能否在 n 个询问内猜出所有节点的权值。如果不能，你的程序应该输出“No”并立刻退出。否则，你的程序应该输出“Yes”，然后开始询问。你可以以任意的大小写形式输出你的答案。例如，“yEs”，“yes”，“Yes”和“YES”都将被视为肯定的回复。

要提出询问，请以“? q u_1 v_1 u_2 v_2 ... u_q v_q ” ($1 \leq q \leq n$ 和 $1 \leq u_i, v_i \leq n$) 的格式一次输出你的所有 q 个询问。在清空输出缓冲区后，你的程序需要读入一行 q 个整数。第 i 个整数表示第 i 个询问的回答。

要猜想权值，请以“! w_2 w_3 ... w_n ” ($0 \leq w_i < 2^{30}$) 的格式输出你的猜测结果。在清空输出缓冲区之后，您的程序应该立即退出。

请注意，每组测试数据的答案都是预先确定的。也就是说，裁判程序并不是适应性的。还请注意，输出猜想不算一次询问。

清空输出缓冲区可以使用以下方式：

- C 和 C++ 使用 `fflush(stdout)`（如果您使用 `printf`）或 `cout.flush()`（如果您使用 `cout`）。
- Java 和 Kotlin 使用 `System.out.flush()`。
- Python 使用 `stdout.flush()`。

Examples

standard input	standard output
4 1 1 2 2 3 2 4 1 3 2	 YES ? 3 1 2 2 3 2 4 ! 1 2 3
5 2 1 2 2 3 3 4 3 5 1 2 3 4	 YES ? 4 1 3 2 4 2 5 4 5 ! 4 5 3 2
6 2 1 2 2 3 3 4 4 5 4 6	 NO

Problem F. 花

“小草，小花，小鸡，小兔子，
谁更会网络流？”
“小花，因为它是 *flower*。”

—熏鸡块君

Frank 痴迷于数字的美。

一天，Frank 正在花园里浇花。看着美丽的花瓣，他想着如果每朵花上都能长出数字就好了。

于是他拿来纸笔，开始涂画他心目中的“数字花”。一张无向连通图被称为“数字花”，当且仅当其满足以下全部三个条件：

1. 如果图中有 n 个节点，则每个节点的标号为 1 到 n 。
2. 图中有恰好 $n - 1$ 条边。任意节点的度数不超过 2，除了 1 号节点。
3. 称与 1 号节点直接相连的节点为关键点。所有关键点的标号两两互质。对于每个非关键点（除了 1 号节点），它的标号都是离其最近的关键点的标号的倍数，且非关键点到最近的关键点的简单路径上的节点标号是单调递减的。

给定一个整数 n ，有多少种不同的包含 n 个点的“数字花”？两张图 G_1 和 G_2 被认为是相同的，当且仅当对于 G_1 中任意的一条边 (u, v) ， G_2 中都存在对应的边 (u, v) 。

由于答案太大，你只需要输出其对质数 p 取模的结果。

Input

每个测试文件仅有一组测试数据。

第一行输入两个正整数 n 和 p ($1 \leq n \leq 10^{10}$, $10^8 < p < 10^9$)。

保证 p 是质数。

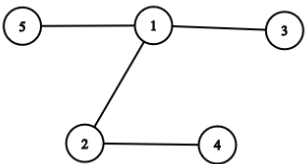
Output

输出一行一个整数，表示不同的“数字花”的数量对 p 取模的结果。

Examples

standard input	standard output
5 998244353	1
10 998244353	4
10000000000 998244353	889033323

Note



样例 1 示意图

对于样例一，只能画出如图所示的一种“数字花”。

Problem G. 最大公因数

有人懂为什么吗？许多出题人都觉得在 G 题放一个主题是 GCD 的题很幽默。

— 卤鸡块君

给定两个整数 a 和 b ，你每轮可以执行以下两种操作之一：

- 若 $a > 0$ ，则令 a 的值减小 $\gcd(a, b)$ 。
- 若 $b > 0$ ，则令 b 的值减小 $\gcd(a, b)$ 。

Grace 想知道使 a 和 b 均变为 0 最少需要的轮数。

[†] $\gcd(x, y)$ 表示 x 和 y 的最大公因数。例如 $\gcd(6, 8) = 2, \gcd(7, 5) = 1$ 。定义 $\gcd(x, 0)$ 和 $\gcd(0, x)$ 的值均为 x 。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 1000$)。每组测试数据的格式如下。

每组数据仅包含一行，行内有两个整数 a 和 b ($1 \leq a \leq b, a \leq 5000, b \leq 10^{18}$)。

在每个测试文件内，保证所有测试数据的 a 之和不超过 10^4 。

Output

对于每组数据，输出一行一个整数，表示将 a 和 b 均变为 0 的最小轮数。

Example

standard input	standard output
3	3
3 4	4
12 20	6
114 514	

Note

对于样例的第一组测试数据，一种可能的最优方案为：

- 对 a 执行操作： $a = 3 - \gcd(3, 4) = 2$ 。
- 对 a 执行操作： $a = 2 - \gcd(2, 4) = 0$ 。
- 对 b 执行操作： $b = 4 - \gcd(0, 4) = 0$ 。

Problem H. 扫描地平线

冷笑话: *radar* 是回文串。
笑点解析: 其实是冷知识, 并不是冷笑话, 搞混了两个相似的概念, 令人忍俊不禁。

—酱鸡块君

Hana 最近需要生产一款雷达, 用于监测她管理的群岛的异常情况。

海洋上有 n 座岛屿, 第 i 座岛屿位于坐标 (x_i, y_i) , 可以被当作平面上的一个点。假设雷达的扫描角度范围为 α , 当雷达旋转到角度 θ 时, 它可以监测到所有位于 $[\theta - \frac{\alpha}{2}, \theta + \frac{\alpha}{2}]$ 角度内的岛屿。

Hana 最近手头紧张, 因此她希望最小化建造雷达的费用。她想知道, 当雷达被放置在坐标原点 $(0, 0)$ 时, 雷达的扫描角度范围 α 最小应该是多少, 才能保证雷达在旋转到任意角度 θ 时, 都能监测到至少 k 个岛屿。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 10^4$)。每组测试数据的格式如下。

每组测试数据的第一行包含两个整数 n 和 k ($1 \leq k \leq n \leq 2 \times 10^5$), 表示岛屿的总数量和雷达至少要监测到的岛屿的数量。

接下来 n 行, 每行两个整数 x_i 和 y_i ($|x_i|, |y_i| \leq 10^9$), 表示一座岛屿的位置。保证任意两个岛屿的坐标互不相同, 且不位于原点。

在每个测试文件内, 保证所有测试数据的 n 之和不超过 2×10^5 。

Output

对于每组数据, 输出一行一个小数, 表示雷达扫描角度范围的最小值, 以弧度制形式输出。

当你的输出与标准答案的绝对误差或相对误差不超过 10^{-6} 时, 你的输出将会被判定为正确。

具体地说, 令你的答案为 a , 标准答案为 b 。你的答案被认为是正确的当且仅当 $\frac{|a-b|}{\max(1, |b|)} \leq 10^{-6}$ 。

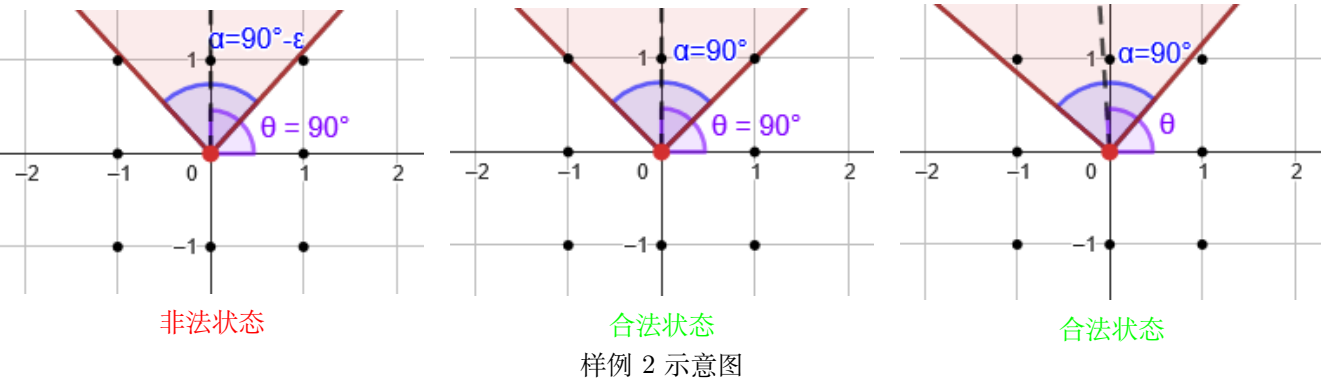
Example

standard input	standard output
5	6.2831853072
1 1	1.5707963268
0 1	5.4977871438
8 2	3.1415926546
1 0	3.1415926536
1 1	
0 1	
-1 1	
-1 0	
-1 -1	
0 -1	
1 -1	
4 2	
-1 1	
0 1	
0 2	
1 1	
4 2	
-10000000000 0	
-998244353 1	
998244353 1	
10000000000 0	
3 1	
0 1	
0 2	
0 -1	

Note

对于样例的第一组测试数据，平面上只有一座岛屿 (0,1)。要保证时刻都能监测到至少一座岛屿，必须将雷达的监测范围设置成 360° ，在弧度制下即为 2π 。

对于样例的第二组测试数据，平面上有 8 座岛屿，任意两座岛屿相隔 45° 。若雷达的范围小于 90° ，当雷达的一个边界刚好不能照射到一座岛屿时，雷达只能监测到一座岛屿（如示意图中左侧的情况）。因此，雷达监测范围的最小值是 90° ，这能保证任意时刻都能监测到至少两座岛屿。



Problem I. 物品

“你的背包，出到现在还没完...”

—奕迅 · 鸡

Ivan 有 n 种物品，每种物品有无限多个。第 i 种物品的重量为 w_i 。

Ivan 想知道，是否可能选出恰好 n 个物品，且选出的物品的总重量恰好为 m 。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 10^4$)。每组测试数据的格式如下。

第一行包含两个整数 n 和 m ($1 \leq n \leq 10^5, 0 \leq m \leq n^2$)。

第二行包含 n 个整数 $w_1, w_2, \dots, w_n$ ($0 \leq w_i \leq n$)。

在每个测试文件内，保证所有测试数据的 n 之和不超过 10^5 。

Output

对于每组数据，如果所求方案存在，请输出 “Yes”；否则，请输出 “No”。

你可以以任意的大小写形式输出你的答案。例如，“yEs”，“yes”，“Yes” 和 “YES” 都将被视为肯定的回复。

Example

standard input	standard output
4	Yes
5 25	No
0 0 0 0 5	No
5 11	No
4 4 4 5 5	
5 0	
1 2 3 4 5	
5 25	
0 1 2 3 4	

Note

在第一组测试数据中，你可以选择 5 个第 5 种物品，总重量为 25，因此你输出 “Yes”。

在第二组测试数据中，可以证明不存在选出总重量为 11 的物品的方案，因此你输出 “No”。

在第三组测试数据中，由于你必须选择 5 个物品，而重量最小的物品的重量为 1，不存在选出总重量为 0 的物品的方案，因此你输出 “No”。

在第四组测试数据中，由于重量最大的物品的重量为 4，不存在选出总重量为 25 的物品的方案，因此你输出 “No”。

Problem J. 又一个排序问题

由题意可知, *Bob* 是寿命更长的那位。

—涮鸡块君

Jessica 是一名排序高手, 精通选择排序、插入排序、冒泡排序和其他许多排序算法。因此, 她决定举办一场排序比赛。

比赛在一个长度为 n 的排列[†] p 上进行, 包含两名参赛选手: Alice 和 Bob。两名选手轮流进行操作, 先手由抛硬币决定。在任意一名玩家操作后, 如果排列是升序的, 则 Alice 立即获胜。如果 Alice 无法在有限轮内获胜, 则认为 Bob 获胜。

当轮到 Alice 操作时, 她可以选择排列的任意两个位置 i, j ($i \neq j, 1 \leq i, j \leq n$), 并交换 p_i 和 p_j 。当轮到 Bob 操作时, 他可以选择排列的相邻两个位置 $i, i+1$ ($1 \leq i < n$), 并交换 p_i 和 p_{i+1} 。双方均不得跳过自己的回合。

给定排列 p 和游戏的先手, 在双方都采取最优策略的情况下, 谁能获得比赛的胜利?

[†] 长度为 n 的排列是一个包含从 1 到 n 的 n 个互不相同整数的数组, 顺序可以是任意的。例如, $[2, 3, 1, 5, 4]$ 是一个排列, 而 $[1, 2, 2]$ 不是排列 (2 在数组中出现了两次), $[1, 3, 4]$ 也不是排列 ($n = 3$, 但数组中有 4)。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 10^4$)。每组测试数据的格式如下。

每组测试数据的第一行包含一个整数 n ($2 \leq n \leq 10^5$) 和一个字符串 s ($s \in \{\text{Alice}, \text{Bob}\}$), 表示排列的长度和第一个进行操作的玩家名。

第二行包含 n 个整数 $p_1, p_2, \dots, p_n$ ($1 \leq p_i \leq n$), 表示排列 p 。保证存在至少一个位置 i , 满足 $p_i \neq i$ 。

在每个测试文件内, 保证所有测试数据的 n 之和不超过 10^5 。

Output

对于每组数据, 输出一行, 包含胜者的名字。若 Alice 将获胜, 输出 “Alice”; 否则, 输出 “Bob”。

Example

standard input	standard output
3	Alice
2 Alice	Bob
2 1	Bob
3 Bob	
1 3 2	
10 Bob	
1 2 3 4 5 6 7 8 10 9	

Problem K. 密钥恢复

你知道吗：在选拔 *ISIJ* 选手的过程中，*ISTJ* 和 *ISFJ* 并不会获得额外加分。

—拌鸡块君

这是一道交互题。

Kevin 不满意自己的 MBTI，因此他找哆啦 A 梦要了一台神奇的机器：MBTI 转换器。

MBTI 转换器一次可以对 8 个人的 MBTI 属性进行转换。它会基于这些人的初始 MBTI 和一个 32 比特的密钥 k ，为每个人生成一个“随机”的新 MBTI。

Kevin 认真研究了这台机器，发现其生成的“随机”MBTI 实际上遵循一定的模式。该机器主要包含三种打乱 MBTI 的基本结构：“xor”、“perm”和“mix”。每种结构均接受 8 个人的 MBTI 作为输入，并会输出 8 个 MBTI 作为结果。在每个结构中，输入的 8 个 MBTI 依次记为 $in_0, in_1, in_2, \dots, in_7$ ，输出的 8 个 MBTI 依次记为 $out_0, out_1, out_2, \dots, out_7$ 。为了方便，Kevin 根据下表将每种 MBTI 用一个十六进制数代替。

INFP	INFJ	INTP	INTJ	ISFP	ISFJ	ISTP	ISTJ
0	1	2	3	4	5	6	7
ENFP	ENFJ	ENTP	ENTJ	ESFP	ESFJ	ESTP	ESTJ
8	9	a	b	c	d	e	f

“xor” 结构：

xor 结构根据密钥 k 对 MBTI 的特定属性进行翻转。

令 k_j 表示密钥 k 的第 j 个二进制位 ($0 \leq j < 32$)，则 xor 结构的功能可以表示为：

$$out_i = in_i \oplus (\overline{k_{4i+3}k_{4i+2}k_{4i+1}k_{4i}})_2$$

其中， $\oplus$ 表示按位异或¹。

“perm” 结构：

perm 结构对 MBTI 进行替换，每种 MBTI 会被替换为另一个固定的 MBTI。

perm 结构的功能可以表示为：

$$out_i = p_{in_i}$$

其中， p 是一个固定的长度为 16 的数组，值如下表所示：

p_0	p_1	p_2	p_3	p_4	p_5	p_6	p_7	p_8	p_9	p_{10}	p_{11}	p_{12}	p_{13}	p_{14}	p_{15}
7	0	5	6	b	f	4	3	1	d	c	e	8	9	a	2

“mix” 结构：

mix 结构用于混合不同人的 MBTI，每个输出由两个输入决定。

mix 结构需要用到一个长度为 8 的中间数组 t ，其定义为：

$$t_i = \begin{cases} in_i \ll 1 & 0 \leq in_i < 8 \\ ((in_i \oplus 8) \ll 1) \oplus 3 & 8 \leq in_i < 16 \end{cases}$$

其中， $\oplus$ 表示按位异或¹， $\ll$ 表示四位二进制数的逻辑左移²。

mix 结构的功能可以表示为：

$$out_i = in_{i \ggg 1} \oplus (t_{(i \ggg 1) \oplus 1})$$

其中， $\oplus$ 表示按位异或¹， $\ggg$ 表示三位二进制数的循环右移³。

初始的 8 个 MBTI，将依次经过 19 个结构，最后得到输出的 8 个 MBTI。这些结构依次是：xor, perm, mix, xor, perm, mix, xor, perm, mix, xor, perm, mix, xor, perm, mix, xor, perm, mix, xor，即连续六轮的“xor, perm, mix”再加一次的“xor”。

使用这台机器的人通常都希望能获得某个特定的 MBTI。可惜的是，哆啦 A 梦在制造这台机器时将密钥 k 封装在了机器内部，且拒绝告诉 Kevin 密钥的值。于是，Kevin 找到了聪明的你。Kevin 希望你构造一些输入（不超过 4096 个，否则机器可能会被损坏），测试运行得到对应的输出，然后从中分析出密钥的值。这样他就可以写一个程序预测每种输入对应的输出，使大家都能得到自己想要的 MBTI。

你能完成这个任务吗？

¹ 按位异或是指将两个二进制数字的每一位进行模 2 意义下的加法运算。例如： $(0011)_2 \oplus (0101)_2 = (0110)_2$ 。

² $x \ll y$ 表示将二进制数 x 逻辑左移 y 位，溢出部分舍弃，空出部分补 0。例如： $(1100)_2 \ll 1 = (1000)_2$, $(1001)_2 \ll 1 = (0010)_2$ 。

³ $x \ggg y$ 表示将二进制数 x 循环右移 y 位。例如： $(101)_2 \ggg 1 = (110)_2$, $(110)_2 \ggg 1 = (011)_2$ 。

Interaction Protocol

要运行 MBTI 转换器，请输出“ $? in_7 in_6 in_5 in_4 in_3 in_2 in_1 in_0$ ” ($in_i \in \{0-9, a-f\}$)，其中 in_i 是第 i 个输入 MBTI 对应的十六进制值。你可以进行这样的操作至多 4096 次。在清空输出缓冲区后，请以“ $out_7 out_6 out_5 out_4 out_3 out_2 out_1 out_0$ ” ($out_i \in \{0-9, a-f\}$) 的格式读入 MBTI 转换器的运行结果。

要回答 k 的值，请输出“ $! k$ ” ($0 \leq k < 2^{32}$)，其中 k 是你认为的密钥 k 的值的十进制表示。你可以进行这样的操作至多一次。在操作后，你应该立即退出你的程序。

如果你运行 MBTI 转换器的次数超过了 4096 次，或是你的输出中包含了非法的字符，交互器将会输出“-1”。在收到这样的回复后，你应该立即退出你的程序，此时你会获得“Wrong Answer”的评测结果。否则，评测结果是不确定的，因为你将尝试从一个已经关闭的输出流中读取数据。

清空输出缓冲区可以使用以下方式：

- C 和 C++ 使用 `fflush(stdout)`（如果您使用 `printf`）或 `cout.flush()`（如果您使用 `cout`）。
- Java 和 Kotlin 使用 `System.out.flush()`。
- Python 使用 `stdout.flush()`。

Example

standard input	standard output
acf7e10b	? 04f37255
	? 9090cfca
105092e0	! 998244353

Note

以下信息可能对解题有所帮助：

1. 输入和输出的 in_i, out_i 之间没有空格，请不要输出多余的空格。输出的内容不是十六进制数，请不要忽略开头可能存在的 0。
2. k 的范围是 $[0, 2^{32} - 1]$ ，在输出答案时，请注意避免输出负数。
3. 本题的下发文件中包含了“interact.hpp”，你可能需要该文件辅助进行本地调试。有关该文件的用法，你可以参考下发文件中的“local_test_example.cpp”。
4. 裁判程序不是自适应的。也就是说，交互器中的 k 在交互开始前就已经确定，不会随着你的交互而改变。
5. 本题共有 64 个测试点。

Problem L. 最后的机会：绝望线缕

出题人你好，我下载了《炉石传说》。为什么我的绝望线缕是三费？

—煎鸡块君

炸鸡块君是炉石传说的忠实玩家。自该游戏在中国大陆恢复运营以来，他一直沉迷于这款游戏，并在标准模式中达到了白银二段。今天，在使用死亡骑士上分时，他遇到了一个强劲的对手炖鸡块君，并被打得只剩 1 血。为了生存下去，炸鸡块君必须消灭炖鸡块君的所有随从。幸运的是，他可以使用法术牌和他的随从的攻击来完成目标。

具体来说，这款游戏中共有两个阵营：炸鸡块君和炖鸡块君。每个阵营都有一些随从。第 i 名随从拥有 h_i 点生命值。当前是炸鸡块君的回合，他的每名随从拥有一次攻击对方阵营的另一名随从的机会。当一名随从攻击另一名随从时，这两名随从都会失去 1 点生命值。当一名随从的生命值降低到 0 点或以下时，它将死亡且无法再攻击或被攻击。



为了达成他的目标，炸鸡块君使用了法术“绝望线缕”，使得每个随从在死亡时触发一次爆炸，爆炸会令所有随从生命值减少 1 点。如果爆炸导致其他随从死亡，其他随从也会立刻爆炸。在所有爆炸效果结束前，炸鸡块君无法令他的随从攻击其他随从。在释放大术后，炸鸡块君可以让自己的每名随从以任意顺序攻击对方的任意一名随从。他希望知道，是否存在一种攻击的顺序，使得炸鸡块君能够消灭炖鸡块君的所有随从。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 5 \times 10^5$)。每组测试数据的格式如下。

第一行包含两个整数 n 和 m ($1 \leq n, m \leq 5 \times 10^5$)，分别表示炸鸡块君的随从数量和炖鸡块君的随从数量。

第二行包含 n 个整数 $h_1, h_2, \dots, h_n$ ($1 \leq h_i \leq 10^9$)，其中 h_i 表示炸鸡块君的第 i 个随从的生命值。

第三行包含 m 个整数 $h'_1, h'_2, \dots, h'_m$ ($1 \leq h'_i \leq 10^9$)，其中 h'_i 表示炖鸡块君的第 i 个随从的生命值。

在每个测试文件内，保证所有测试数据的 n 之和不超过 5×10^5 ，所有测试数据的 m 之和不超过 5×10^5 。

Output

对于每组数据，如果炸鸡块君能够消灭炖鸡块君的所有随从，请输出 “Yes”；否则，请输出 “No”。

你可以以任意的大小写形式输出你的答案。例如，“yEs”，“yes”，“Yes” 和 “YES” 都将被视为肯定的回复。

Examples

standard input	standard output
3 3 2 1 1 4 2 6 3 2 1 1 4 2 7 2 1 100 100 2	Yes No Yes
3 7 1 1 1 1 1 1 1 1 9 5 2 3 4 5 6 7 1 6 5 3 3 4 5 6 7 1 5 7	No No Yes

Note

在样例一的第一组测试数据中，一种可能的方案是：炸鸡块君的第 3 个随从攻击炖鸡块君的第 2 个随从；炸鸡块君的第 2 个随从攻击炖鸡块君的第 2 个随从。此时，由于炸鸡块君的第 2 个随从死亡，将会引发一次爆炸。该爆炸又会导致其他随从死亡，进而继续引发爆炸。最终，所有随从都被消灭了。

在样例一的第三组测试数据中，一种可能的方案是：炸鸡块君的第 1 个随从攻击炖鸡块君的第 1 个随从；炸鸡块君的第 2 个随从攻击炖鸡块君的第 1 个随从。此时，炖鸡块君的第 1 个随从死亡，引发一次爆炸。最终，炸鸡块君剩下了两个随从，每个随从的生命值为 98，并消灭了炖鸡块君的所有随从。炸鸡块君成功存活了下来。

Problem M. 矩阵构造

构造矩阵、样例里有组 “2 3”.....
我们最近做过这样的题吗？

—炖鸡块君

Mary 喜欢构造矩阵！

今天，Mary 想要将一个长度为 $n \times m$ 的排列[†] 填入一个 $n \times m$ 的矩阵 A 中，使得任意相邻两个元素的和都不重复。

换句话说，对于任意的 $1 \leq x_1, x_2, x_3, x_4 \leq n, 1 \leq y_1, y_2, y_3, y_4 \leq m$ ，若以下条件全部满足：

- $x_2 \geq x_1, y_2 \geq y_1, x_4 \geq x_3, y_4 \geq y_3$ ；
- $|x_2 - x_1| + |y_2 - y_1| = 1, |x_4 - x_3| + |y_4 - y_3| = 1$ ；
- $(x_1, y_1) \neq (x_3, y_3)$ 或 $(x_2, y_2) \neq (x_4, y_4)$ ；

则：

$$A_{x_1, y_1} + A_{x_2, y_2} \neq A_{x_3, y_3} + A_{x_4, y_4}$$

例如，当 $n = 2$ 且 $m = 3$ 时，矩阵 B 是一种可行的方案，而矩阵 C 不可行，因为 $C_{1,1} + C_{2,1} = C_{1,2} + C_{1,3}$ 。

$$B = \begin{bmatrix} 1 & 3 & 2 \\ 6 & 5 & 4 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

给定 n 和 m ，上述要求能否被满足？如果能，请输出一种可行的方案。

[†] 长度为 n 的排列是一个包含从 1 到 n 的 n 个互不相同整数的数组，顺序可以是任意的。例如， $[2, 3, 1, 5, 4]$ 是一个排列，而 $[1, 2, 2]$ 不是排列（2 在数组中出现了两次）， $[1, 3, 4]$ 也不是排列（ $n = 3$ ，但数组中有 4）。

Input

每个测试文件包含多组测试数据。第一行包含测试数据的组数 T ($1 \leq T \leq 10^4$)。每组测试数据的格式如下。

第一行包含两个整数 n, m ($1 \leq n, m \leq 1000$)。

在每个测试文件内，保证所有测试数据的 $n \times m$ 之和不超过 10^6 。

Output

对于每组数据，在第一行输出是否存在可行的方案。如果存在可行的方案，输出 “Yes”；否则，输出 “No”。你可以以任意的大小写形式输出你的答案。例如，“yEs”，“yes”，“Yes” 和 “YES” 都将被视为肯定的回复。

如果存在可行的方案，你还需要输出 n 行，每行 m 个整数。第 i 行的第 j 个数表示矩阵的第 i 行第 j 列上的数字。你需要保证输出的数字集合构成一个长度为 $n \times m$ 的排列。如果有多种可行的方案，你可以输出任何一种。

Example

standard input	standard output
2	yEs
1 1	1
2 3	YES
	1 3 2
	6 5 4